

# GitHub Copilot Custom Agents Tutorial

---

カスタムエージェント設計と実装を段階的に学ぶハンズオン

2026-04-03



# GitHub Copilot Custom Agents

## カスタムエージェント実践チュートリアル

GitHub Copilot の Custom Agents を題材に、役割設計、プロンプト制御、ツール制限、運用ルールを 段階的に学ぶ日本語ハンズオン教材です。

[Note] ブラウザで <https://duwenji.github.io/spa-quiz-app/> を開くと、関連トピックをクイズ形式で復習できます。

- 著者: 杜 文吉
- 対象: Copilot 活用を高度化したい開発者 / テックリード
- テーマ: `.agent.md` / `instructions` / `skills` / governance

この教材で学べること - `Custom Agents` の役割と設計原則 - `.agent.md` の構造と最小ツール設計 - `instructions` / `skills` / `prompts` の使い分け - チームで安全に共有・改善する運用方法

# 目次

---

- 第0章 Fundamentals (#chapter-00-fundamentals)
  - 0.0 Part 0: `.agent.md` の位置づけ (#section-00-fundamentals-00-agent-format-overview)
  - 0.4 Part 0-4: Customization Map (#section-00-fundamentals-04-customization-map)
- 第1章 Basics (#chapter-01-basics)
  - 1.1 Part 1-1: カスタムエージェントとは (#section-01-basics-01-introduction)
  - 1.2 Part 1-2: Skills との違い (#section-01-basics-02-vs-skills)
  - 1.3 Part 1-3: カスタムエージェントの動き方 (#section-01-basics-03-how-custom-agents-work)
- 第2章 Comparison (#chapter-02-comparison)
  - 2.1 Part 2-1: 通常プロンプトとの違い (#section-02-comparison-01-vs-prompts)
  - 2.2 Part 2-2: Skills / MCP との違い (#section-02-comparison-02-vs-skills-and-mcp)
  - 2.3 Part 2-3: どれを選ぶべきか (#section-02-comparison-03-selection-guide)
- 第3章 Implementation (#chapter-03-implementation)
  - 3.1 Part 3-1: 最初のエージェントを作る (#section-03-implementation-01-getting-started)
  - 3.2 Part 3-2: エージェント定義の読み方 (#section-03-implementation-02-agent-structure)

- 3.3 Part 3-3: サンプル1 - Docs Writer Agent (#section-03-implementation-03-sample-docs-agent)
- 3.4 Part 3-4: サンプル2 - Debug Helper Agent (#section-03-implementation-04-sample-debug-agent)
- 3.5 Part 3-5: サンプル3 - Repo Maintainer Agent (#section-03-implementation-05-sample-repo-agent)
- 3.6 Part 3-6: Agent をどう評価するか (#section-03-implementation-06-agent-evaluation)
- 3.7 Part 3-7: サンプル4 - Release Prep + Skill 連携 (#section-03-implementation-07-sample-release-prep-agent)
- 第4章 Operations (#chapter-04-operations)
  - 4.1 Part 4-1: ベストプラクティスとアンチパターン (#section-04-operations-01-best-practices)
  - 4.2 Part 4-2: Agent レビュー・チェックリスト (#section-04-operations-02-review-checklist)
  - 4.3 Part 4-3: Packaging と公開準備 (#section-04-operations-03-packaging-and-publishing)
  - 4.4 Part 4-4: 公開前チェックリスト (#section-04-operations-04-release-checklist)
  - 4.5 Part 4-5: FAQ (#section-04-operations-05-faq)
- 第5章 Advanced Topics (#chapter-05-advanced-topics)
  - 5.1 Part 5-1: Subagents と Handoffs (#section-05-advanced-topics-01-subagents-and-handoffs)
  - 5.2 Part 5-2: チーム向け設計パターン (#section-05-advanced-topics-02-team-design-patterns)

- 5.3 Part 5-3: Orchestrator Agent の例 (#section-05-advanced-topics-03-orchestrator-example)



# 第0章 Fundamentals

---

## 0.0 Part 0: `.agent.md` の位置づけ

---

### 0.0.1 概要

GitHub Copilot の **カスタムエージェント**は、特定の役割に合わせて振る舞いを定義した専用アシスタントです。

通常の都度プロンプトよりも、**役割・境界・利用ツール**を安定して共有しやすいのが特徴です。

---

### 0.0.2 どこに置くのか

ワークスペース単位では、通常次の場所に配置します。

```
.github/agents/*.agent.md
```

例:

```
.github/agents/docs-writer.agent.md  
.github/agents/debug-helper.agent.md
```

---

### 0.0.3 何を定義するのか

`.agent.md` では主に以下を定義します。



| 項目                       | 役割                          |
|--------------------------|-----------------------------|
| <code>description</code> | いつ使う agent かを示す説明           |
| <code>tools</code>       | 使ってよいツールの範囲（= 利用できる機能・権限）   |
| <code>model</code>       | 必要に応じて使いたいモデル               |
| <code>agents</code>      | 呼び出せる subagent の制御（必要な場合のみ） |
| 本文                       | 役割、制約、手順、出力形式               |

`agents` は **任意** です。利用可能な agent 名は環境依存なので、必要なければ省略して問題ありません。

#### 0.0.4 `tools` は何を表すのか

`tools` は、Custom Agent 自身が `.agent.md` の中で実装する関数名ではありません。

これは GitHub Copilot / VS Code 側が提供するツールランタイムのうち、その agent に使わせてよい機能・権限を宣言する設定 です。

たとえば `tools: [read, edit, search]` は、「この agent はファイルを読み、検索し、必要に応じて編集してよい」という意味になります。

0.0.4.1 よく使う `tools` の例

| <code>tools</code> の例 | できること           | 向く場面              |
|-----------------------|-----------------|-------------------|
| <code>read</code>     | ファイルや内容を読む      | 仕様確認、ドキュメント読解     |
| <code>search</code>   | ワークスペース内を検索する   | 関連箇所の調査、影響範囲確認    |
| <code>edit</code>     | ファイルを編集する       | ドキュメント修正、軽微なコード変更 |
| <code>execute</code>  | コマンドを実行して確認する   | デバッグ、ビルド、検証       |
| <code>todo</code>     | 作業の手順を整理する      | 複数ステップの作業管理       |
| <code>agent</code>    | 別の subagent を呼ぶ | 役割分担、調査の切り分け      |

この一覧は代表例です。実際に利用できる `tools` は **環境やバージョンによって異なる** 場合があります。

内部では `read_file` のような具体的なツールにマッピングされますが、`.agent.md` では **カテゴリとしての許可範囲** を書けば十分です。

## 0.0.5 最小サンプル

```
(#cb3-1)---
(#cb3-2)description: "Use this agent when improving README files or
      markdown documentation."
(#cb3-3)tools: [read, edit, search]
(#cb3-4)user-invocable: true
(#cb3-5)---
(#cb3-6)
(#cb3-7)You are a documentation-focused Copilot agent.
(#cb3-8)
(#cb3-9)## Constraints
(#cb3-10)- Keep edits concise
(#cb3-11)- Preserve technical accuracy
(#cb3-12)- Do not modify unrelated files
```

## 0.0.6 他のカスタマイズとの違い

| 種類                                   | 向いている用途              |
|--------------------------------------|----------------------|
| <code>copilot-instructions.md</code> | リポジトリ全体に共通する指示       |
| <code>*.prompt.md</code>             | 単発の定型依頼              |
| <code>SKILL.md</code>                | 再利用できる作業手順           |
| <code>*.agent.md</code>              | 役割・制約・ツールを持つ専用エージェント |

迷ったら、全体ルールは `instructions`、単発テンプレは `prompt`、手順化は `skill`、役割化は `agent` と考えると整理しやすいです。

## 0.4 Part 0-4: Customization Map

### 0.4.1 まず全体像をつかむ

GitHub Copilot のカスタマイズ要素は、似て見えても役割が異なります。

| 要素                                   | 役割            | 向く場面             |
|--------------------------------------|---------------|------------------|
| <code>copilot-instructions.md</code> | リポジトリ全体の共通ルール | いつも守ってほしい方針      |
| <code>*.prompt.md</code>             | 単発の定型依頼       | その都度使うテンプレート     |
| <code>SKILL.md</code>                | 再利用可能な手順      | ビルド、生成、検証など      |
| <code>*.agent.md</code>              | 役割を持つ専用 agent | docs担当、debug担当など |
| <code>MCP</code>                     | 外部システムやツール接続  | API、DB、社内情報参照    |

---

## 0.4.2 関係イメージ

```
Repository rules
  copilot-instructions.md

Reusable task templates
  prompts
  skills

Role-based assistants
  custom agents

External capabilities
  MCP
```

---

## 0.4.3 覚え方

- 全体ルール -> instructions
- 単発テンプレ -> prompt
- 手順の再利用 -> skill
- 役割の分離 -> agent
- 外部接続 -> MCP

この 5 つを分けて考えると、設計が整理しやすくなります。

# 第1章 Basics

---

## 1.1 Part 1-1: カスタムエージェントとは

---

### 1.1.1 カスタムエージェントが解決する問題

通常の Copilot 利用では、次のような悩みが起きやすくなります。

- 毎回同じ指示を書き直している
- 作業ごとに期待する出力形式がぶれる
- 使ってほしいツールと、使ってほしくないツールの境界が曖昧
- チームで共通のやり方を再利用しづらい

カスタムエージェントは、これらを **役割定義** と **ツール制御** で整理します。

---

### 1.1.2 一言でいうと

カスタムエージェントは、「この役割のときは、こう振る舞う」をファイルとして持てる仕組みです。

たとえば次のような専用 agent を用意できます。

- ドキュメント整備担当
- デバッグ担当
- リポジトリ保守担当
- 学習教材レビュー担当

---

### 1.1.3 主なメリット

#### 1. 再利用しやすい

- 毎回プロンプトを書き直さなくてよい

#### 2. 責務を分けやすい

- ドキュメント作業とデバッグ作業を分離できる

#### 3. 安全性を高めやすい

- 利用ツールを最小限に絞れる

#### 4. チームで共有しやすい

- `.github/agents/` に置いてレビューできる
- 

### 1.1.4 設計の基本姿勢

- 1 agent = 1つの中心的な役割
- `description` は具体的に書く
- `tools` は必要最小限にする
- 期待する出力を本文で明確にする

この4つを守るだけでも、使い勝手はかなり安定します。

## 1.2 Part 1-2: Skills との違い

---

### 1.2.1 先に結論

- Skill は「再利用できる手順」

- Custom Agent は「役割を持つ専用アシスタント」

似ていますが、設計の発想が少し違います。

### 1.2.2 比較表

| 観点     | Custom Agent                                         | Skill                          |
|--------|------------------------------------------------------|--------------------------------|
| ねらい    | 役割と振る舞いを固定する                                         | 作業手順を再利用する                     |
| 主な置き場所 | <code>.github/agents/</code>                         | <code>.github/skills/</code>   |
| 中心要素   | <code>description</code> , <code>tools</code> , 役割定義 | <code>SKILL.md</code> の手順や参考資料 |
| 向く用途   | docs担当、debug担当、review担当                              | テスト生成、ビルド手順、分析フロー              |
| 重要設計   | 単一責務、ツール制約、境界                                        | 手順の明確さ、入力条件、補助資料               |

### 1.2.3 どう使い分けるか

#### 1.2.3.1 Custom Agent が向く例

- 「README 改善専用の agent がほしい」
- 「バグ調査のときだけ terminal を使えるようにしたい」
- 「教材レビュー用の専門ロールを作りたい」



### 1.2.3.2 Skill が向く例

- 「毎回同じ検証フローを実行したい」
  - 「eBook ビルドの共通手順をまとめたい」
  - 「クイズ生成のプロセスを再利用したい」
- 

### 1.2.4 迷ったときの判断基準

次の質問で決めるとわかりやすいです。

1. 役割を分けたいのか？ -> Agent
2. 作業手順を再利用したいのか？ -> Skill
3. リポジトリ全体のルールを書きたいのか？ -> copilot-instructions.md
4. 単発テンプレを残したいのか？ -> prompt

## 1.3 Part 1-3: カスタムエージェントの動き方

---

### 1.3.1 基本フロー

カスタムエージェントは、おおむね次の流れで使われます。

ユーザーの依頼

down

説明文（`description`）に合う agent が選ばれる

down

agent の役割・制約・利用可能ツールが適用される

down

必要ならファイル読取・検索・編集・実行を行う

down

結果をまとめて返す

---

### 1.3.2 `description` が重要な理由

`description` は、いつその agent を使うべきかを示す入口です。

悪い例:

```
(#cb6-1)description: "Helpful assistant"
```

良い例:

```
(#cb7-1)description: "Use this agent when updating README files,  
markdown guides, or tutorial content."
```

具体的なキーワードがあるほど、役割がぶれにくくなります。

---

### 1.3.3 `tools` は絞るほど強い

`tools` は agent の中で自作する関数一覧ではなく、GitHub Copilot / VS Code が提供する機能のうち、何を使ってよいかを宣言する設定です。

ツールを増やしすぎると、agent の責務が曖昧になります。

たとえば docs 専用なら:

```
(#cb8-1)tools: [read, edit, search]
```

デバッグ専用なら:

```
(#cb9-1)tools: [read, search, execute]
```

まずは **最小ツールセット** で始め、必要になったら増やすのが安全です。

### 1.3.4 本文には何を書くか

本文では次を明確にします。

- **Role:** 何担当か
- **Constraints:** 何をしてはいけないか
- **Approach:** どう進めるか
- **Output Format:** どう返すか

この4点があると、agent の挙動がかなり安定します。

# 第2章 Comparison

---

## 2.1 Part 2-1: 通常プロンプトとの違い

---

### 2.1.1 先に結論

- 通常プロンプト: その場の依頼を書く
  - Custom Agent: 役割・制約・ツールを再利用できる形で定義する
- 

### 2.1.2 何が違うのか

| 観点    | 通常プロンプト   | Custom Agent                         |
|-------|-----------|--------------------------------------|
| 再利用性  | 低い        | 高い                                   |
| 役割の固定 | 毎回書く必要がある | ファイルで共有できる                           |
| ツール制御 | その都度あいまい  | 明示しやすい                               |
| チーム共有 | 難しい       | <code>.github/agents/</code> で共有しやすい |
| 向く場面  | 単発の依頼     | 繰り返し発生する役割型作業                        |

---

### 2.1.3 通常プロンプトが向く場面

- 1回だけの質問
- すぐ終わる単発相談

- まだ役割が定まっていない作業

## 2.1.4 Custom Agent が向く場面

- README 改善を何度も行う
  - バグ調査の進め方を標準化したい
  - チームで同じ観点のレビューをしたい
- 

## 2.1.5 実務上の判断

同じ指示を 3回以上書いているなら、agent 化を検討する価値があります。

## 2.2 Part 2-2: Skills / MCP との違い

---

### 2.2.1 ざっくり整理

- **Skill:** 作業手順をまとめる
- **Custom Agent:** 役割と振る舞いを持たせる
- **MCP:** 外部ツールやデータにアクセスする仕組み

### 2.2.2 比較表

| 観点   | Custom Agent                 | Skill                        | MCP                                |
|------|------------------------------|------------------------------|------------------------------------|
| 主目的  | 役割の定義                        | 手順の再利用                       | 能力や外部接続の提供                         |
| 置き場所 | <code>.github/agents/</code> | <code>.github/skills/</code> | MCP server / config                |
| 中心要素 | persona, constraints, tools  | procedure, docs, assets      | tools, resources, external systems |
| 向く例  | docs担当、debug担当               | build手順、生成フロー                | DB検索、API操作、社内情報参照                  |

### 2.2.3 組み合わせもできる

実務では、これらは競合ではなく**補完関係**です。

例: - Agent が役割を担当する - Skill が標準手順を提供する - MCP が外部情報やツール実行を支える

---

## 2.2.4 判断基準

1. 役割を分けたい -> `Custom Agent`
2. 毎回同じ手順を再利用したい -> `Skill`
3. 外部システムに触れたい -> `MCP`

## 2.3 Part 2-3: どれを選ぶべきか

---

### 2.3.1 シンプルな判断フロー

共通ルールを書きたい？

はい -> ``copilot-instructions.md``

単発テンプレを残したい？

はい -> ``prompt``

役割を持つ専用アシスタントがほしい？

はい -> ``Custom Agent``

標準手順を再利用したい？

はい -> ``Skill``

外部システムやAPIに触れたい？

はい -> ``MCP``

---

### 2.3.2 よくある組み合わせ

#### 2.3.2.1 パターン1: 小さく始める

- `copilot-instructions.md`
- `docs-writer.agent.md`

### 2.3.2.2 パターン2: 手順も共有する

- `copilot-instructions.md`
- `debug-helper.agent.md`
- `SKILL.md`

### 2.3.2.3 パターン3: 実務連携まで広げる

- `copilot-instructions.md`
  - `repo-maintainer.agent.md`
  - `MCP`
- 

## 2.3.3 おすすめの始め方

初学者なら次の順が安全です。

1. `copilot-instructions.md`
2. 小さな `Custom Agent`
3. 必要なら `Skill`
4. 最後に `MCP`



# 第3章 Implementation

---

## 3.1 Part 3-1: 最初のエージェントを作る

---

### 3.1.1 ゴール

ここでは、最小構成のカスタムエージェントを 1 つ作ります。

---

### 3.1.2 Step 1: ファイルを作る

次の場所にファイルを作成します。

```
.github/agents/my-first-docs.agent.md
```

---

### 3.1.3 Step 2: 最小 frontmatter を書く

```
(#cb12-1)---
(#cb12-2)description: "Use this agent when improving README files or
      markdown documentation."
(#cb12-3)tools: [read, edit, search]
(#cb12-4)user-invocable: true
(#cb12-5)---
```

ここで大切なのは次の3つです。

- `description`: いつ使うかを具体的に書く
- `tools`: 最小限に絞る

- `user-invocable` : 手動で使えるようにする
- 

### 3.1.4 Step 3: 本文を書く

```
(#cb13-1)You are a documentation-focused Copilot agent.
(#cb13-2)
(#cb13-3)## Constraints
(#cb13-4)- Keep edits concise
(#cb13-5)- Preserve technical accuracy
(#cb13-6)- Do not modify unrelated files
(#cb13-7)
(#cb13-8)## Approach
(#cb13-9)1. Read the target file
(#cb13-10)2. Improve clarity and structure
(#cb13-11)3. Summarize the changes
```

### 3.1.5 Step 4: 試す

例:

- “README を読みやすく整えて”
  - “この教材の説明を初学者向けに短くして”
  - “Markdown の見出し構造を改善して”
- 

### 3.1.6 最初のチェックリスト

- ☐ `description` が具体的か
- ☐ `tools` を盛りすぎているか
- ☐ してはいけないことを書いているか

□ 返してほしい出力の形が明確か

最初は完璧を目指さず、1 role / 1 task に絞るのが成功のコツです。

## 3.2 Part 3-2: エージェント定義の読み方

---

### 3.2.1 典型的な構成

```
(#cb14-1)---
(#cb14-2)description: "Use this agent when investigating bugs, runtime
      errors, or broken builds."
(#cb14-3)tools: [read, search, execute]
(#cb14-4)user-invocable: true
(#cb14-5)---
(#cb14-6)
(#cb14-7)You are a debugging-focused Copilot agent.
(#cb14-8)
(#cb14-9)## Role
(#cb14-10)- Investigate failures
(#cb14-11)- Trace root cause
(#cb14-12)
(#cb14-13)## Constraints
(#cb14-14)- Do not guess
(#cb14-15)- Verify before claiming success
(#cb14-16)
(#cb14-17)## Approach
(#cb14-18)1. Read the error
(#cb14-19)2. Search relevant files
(#cb14-20)3. Verify the cause
(#cb14-21)
(#cb14-22)## Output Format
(#cb14-23)- Summary
(#cb14-24)- Root cause
(#cb14-25)- Fix
(#cb14-26)- Evidence
```

---

### 3.2.2 frontmatter の見方

| 項目             | 意味                 |
|----------------|--------------------|
| description    | いつ使うかを示す説明         |
| tools          | 使ってよいツール           |
| user-invocable | 手動選択できるか           |
| model          | 必要に応じて使いたいモデル      |
| agents         | 呼び出せる subagent の制御 |

---

### 3.2.3 本文で差が出るポイント

#### 3.2.3.1 Role

この agent の中心責務を短く定義します。

#### 3.2.3.2 Constraints

「何をしないか」を明示します。

#### 3.2.3.3 Approach

手順を 3～5 ステップ程度に絞ります。

#### 3.2.3.4 Output Format

返答の形を固定すると、利用者が扱いやすくなります。

---

### 3.2.4 よくある失敗

- `description` が曖昧
- `tools` を入れすぎる
- 役割が複数混ざる
- 出力形式が毎回ぶれる

## 3.3 Part 3-3: サンプル1 - Docs Writer Agent

---

### 3.3.1 目的

Markdown や README の改善を専門にする最小構成の agent です。

---

### 3.3.2 サンプル

参照ファイル: `.github/agents/docs-writer.agent.md`

```
(#cb15-1)---
(#cb15-2)description: "Use this agent when updating README files,
      markdown documentation, guides, tutorials, or learning
      content."
(#cb15-3)tools: [read, edit, search]
(#cb15-4)user-invocable: true
(#cb15-5)---
```

---

### 3.3.3 この agent が向く場面

- README の見出し整理
- 教材の文体統一

- Markdown の流れ改善
  - 初学者向けのわかりやすい表現への修正
- 

### 3.3.4 設計のポイント

1. `read`, `edit`, `search` だけに限定
  2. コード変更は原則しない
  3. 説明は短く、学習しやすさを優先
- 

### 3.3.5 学び

このサンプルは、**1 role に絞ることの重要性**を示しています。最初の custom agent は、このくらい小さく始めるのが適切です。

## 3.4 Part 3-4: サンプル2 - Debug Helper Agent

---

### 3.4.1 目的

エラー調査、失敗原因の切り分け、検証を支援する agent です。

---

### 3.4.2 サンプル

参照ファイル: `.github/agents/debug-helper.agent.md`

```
(#cb16-1)---
(#cb16-2)description: "Use this agent when investigating bugs, runtime
      errors, broken builds, failing tests, or root-cause
      analysis."
(#cb16-3)tools: [read, search, execute]
(#cb16-4)user-invocable: true
(#cb16-5)---
```

---

### 3.4.3 この agent が向く場面

- テスト失敗の原因調査
- ビルドエラーの切り分け
- 実行時例外の確認
- 根本原因を整理したいとき

---

### 3.4.4 設計のポイント

1. まず再現・確認を重視する
2. 推測より証拠を優先する
3. 成功を断定する前に検証する
4. 一度に複数の思いつき修正を重ねない

---

### 3.4.5 学び

debug 系 agent は便利ですが、権限を広げすぎると危険です。そのため `tools` を絞り、**verification-first** をルールとして明記します。

## 3.5 Part 3-5: サンプル3 - Repo Maintainer Agent

---

### 3.5.1 目的

リポジトリ全体の整合性を保ち、軽微な保守作業を安全に進めるための agent です。

---

### 3.5.2 サンプル

参照ファイル: `.github/agents/repo-maintainer.agent.md`

この agent は、次のような作業に向いています。

- ドキュメント構成の整理
  - 学習教材の見出し統一
  - 関連ファイルの軽微なメンテナンス
  - multi-step 作業の見える化
- 

### 3.5.3 設計のポイント

1. 作業対象を広げすぎない
  2. 大規模リファクタではなく、小さな整備に集中する
  3. `todo` を使って進捗を可視化する
  4. 検証していない変更は断定しない
- 

### 3.5.4 学び

`repo-maintainer` は便利ですが、何でも屋にしないことが重要です。広すぎる責務を避ける設計例として参考になります。



## 3.6 Part 3-6: Agent をどう評価するか

---

### 3.6.1 なぜ評価が必要か

カスタムエージェントは、ファイルを作っただけでは品質が保証されません。

確認したいのは、次のような点です。

- 期待した場面で使いやすいか
  - 出力が毎回ぶれすぎないか
  - 危険な権限の使い方をしていないか
  - 役割が明確で、他 agent と混ざっていないか
- 

### 3.6.2 最低限の評価観点

| 観点     | 見るポイント                              |
|--------|-------------------------------------|
| 発見しやすさ | <code>description</code> を見て用途が伝わるか |
| 一貫性    | 出力形式が安定しているか                        |
| 安全性    | 不要なツール権限がないか                        |
| 実用性    | 実際の依頼で役立つか                          |

---

### 3.6.3 まずやるべき評価方法

1. 代表的な依頼を3～5個用意する
2. その依頼で agent を使ってみる
3. 出力の良し悪しを記録する

#### 4. `description` や `constraints` を微調整する

---

### 3.6.4 改善サイクル

```
作成
  down
試す
  down
ぶれや失敗を記録
  down
説明文・制約・出力形式を調整
  down
再度試す
```

### 3.6.5 重要な考え方

agent の品質は、一回で完成するものではなく、使いながら磨くものです。そのため、小さく始めて改善する姿勢が重要です。

## 3.7 Part 3-7: サンプル4 - Release Prep + Skill 連携

---

### 3.7.1 目的

`release-prep.agent.md` と `release-readiness/SKILL.md` を組み合わせて、「役割を持つ Agent」と「再利用できる手順としての Skill」をどう分けるかを学ぶためのサンプルです。

---

### 3.7.2 参照ファイル

- `.github/agents/release-prep.agent.md`

- `.github/skills/release-readiness/SKILL.md`

---

### 3.7.3 このサンプルが向く場面

- 教材リポジトリの公開前チェック
- `README.md` や `CHANGELOG.md` の整合性確認
- `ebook` や `quiz` の生成結果の最終確認
- 「公開可能か / 修正が必要か」を短く整理したい場面

---

### 3.7.4 なぜ Agent と Skill を分けるのか

このサンプルでは、役割と手順を次のように分離しています。

| 要素                                      | 担当すること                        |
|-----------------------------------------|-------------------------------|
| <code>release-prep.agent.md</code>      | 公開準備の調整役として、何を確認すべきかを整理する     |
| <code>release-readiness/SKILL.md</code> | 公開前の標準手順やチェックリストを再利用可能な形で提供する |

つまり、

- **Agent** = 誰として振る舞うか
- **Skill** = どう進めるか

を分けているのがポイントです。

---

### 3.7.5 設計のポイント

1. Agent 側に全手順を書き込みすぎない
  2. 繰り返し使う確認手順は Skill に寄せる
  3. `description` に「公開前チェック」「リリース準備」などの自然な語を入れる
  4. 出力形式を固定して、結果を比較しやすくする
- 

### 3.7.6 例: 想定される依頼

- 「この教材リポジトリの公開前チェックをして」
- 「README と生成物の最終確認をしたい」
- 「リリース前に修正点があるか見て」

このような依頼に対して、Agent が全体を整理し、必要な標準フローは Skill で補う設計です。

---

### 3.7.7 実演プロンプト例

以下のような依頼文で試すと、このサンプルの狙いが分かりやすくなります。

この教材リポジトリの公開前チェックをして

README と CHANGELOG を見て、リリース前に不足がないか確認して

ebook と quiz の生成結果も含めて、公開可能か判断して

ポイントは、

- **Agent** が全体の役割と判断を担当する
- **Skill** が繰り返し使う確認手順を支える

という分担が自然に見えるかどうかです。

---

### 3.7.8 学び

このサンプルは、**Custom Agent** と **Skill** は競合ではなく補完関係であることを示しています。

- 役割を明確にしたい -> **Agent**
- 手順を再利用したい -> **Skill**

実務では、この2つを組み合わせると、**使いやすく、保守しやすいカスタマイズ**にしやすいくなります。

# 第4章 Operations

---

## 4.1 Part 4-1: ベストプラクティスとアンチパターン

---

### 4.1.1 ベストプラクティス

#### 4.1.1.1 単一責務にする

- 1つの agent に 1つの中心的役割を持たせる
- docs 用、debug 用、review 用を分ける

#### 4.1.1.2 `description` を具体的にする

- 「いつ使うか」を文章で書く
- README、debug、broken build などのキーワードを含める

#### 4.1.1.3 `tools` は最小限にする

- まず read-only に近い構成から始める
- 実行や編集は必要な agent にだけ許可する

#### 4.1.1.4 出力形式を固定する

- summary
- root cause
- next action
- evidence

#### 4.1.1.5 Agent と Skill を混ぜすぎない

- Agent は**役割**に集中させる
  - Skill は**再利用手順**に集中させる
  - 毎回同じ確認フローを書くなら Skill 側へ寄せる
- 

### 4.1.2 アンチパターン

#### 4.1.2.1 Swiss-army agent

何でもできる 1体にしようとして、責務が曖昧になるパターンです。

#### 4.1.2.2 曖昧な description

"Helpful assistant" のような説明では役割が伝わりません。

#### 4.1.2.3 ツール過多

最初から `execute` , `edit` , `web` , `agent` を全部入れると、設計がぶれやすくなります。

#### 4.1.2.4 検証なしの完了宣言

特に debug 系 agent で危険です。実行結果やテスト確認を伴うべきです。

---

### 4.1.3 推奨レビュー観点

- この agent の責務は一文で説明できるか
- 許可ツールは本当に必要最小限か
- 使う場面と使わない場面が明確か
- 出力形式が利用者にとって一貫しているか

## 4.2 Part 4-2: Agent レビュー・チェックリスト

---

### 4.2.1 仕様レビュー

- ☐ `description` に具体的な利用場面が書かれている
- ☐ `tools` が最小限に絞られている
- ☐ 役割が 1 文で説明できる
- ☐ やってはいけないことが書かれている

### 4.2.2 運用レビュー

- ☐ 利用者がどの agent を選べばよいか迷わない
- ☐ 出力形式が一貫している
- ☐ 危険な権限を安易に与えていない
- ☐ 変更後の確認方法が決まっている

### 4.2.3 改善レビュー

- ☐ 実際の利用例に基づいて更新している
- ☐ 形だけでなく役に立つ agent になっている
- ☐ 責務が広がりすぎていない

## 4.3 Part 4-3: Packaging と公開準備

---

### 4.3.1 この教材リポジトリの方針

このリポジトリでは、以下の形で再利用しやすくすることを想定しています。

- Markdown ベースの教材として読む
- eBook としてビルドする



- GitHub Pages で公開する
- 

### 4.3.2 用意したファイル

- `.github/skills-config/ebook-build/`
- `.github/workflows/validate.yml`
- `.github/workflows/pages.yml`
- `package.json`

これにより、教材の build と公開導線をシンプルに保てます。

---

### 4.3.3 基本コマンド

```
(#cb21-1)git submodule update --init --recursive
(#cb21-2)npm install
(#cb21-3)npm run ebook:build
```

実行前に、共有スキル群 ( `shared-copilot-skills` ) を `submodule` として初期化しておくのが推奨です。

---

### 4.3.4 公開前の確認

- 章番号がそろっているか
- README のリンク切れがないか
- `.agent.md` サンプルの説明文が具体的か
- eBook / cover 出力先が `.gitignore` 対象になっているか

---

### 4.3.5 GitHub Pages 公開

この教材では `docs/` を GitHub Pages で公開できるようにしています。

- workflow: `.github/workflows/pages.yml`
- site entry: `docs/index.md`
- Jekyll config: `docs/_config.yml`

想定 URL:

```
https://duwenji.github.io/github-copilot-custom-agents-tutorial/
```

---

### 4.3.6 小さく公開するコツ

最初から完成版を目指すより、**初版** -> **フィードバック** -> **改善版** の流れで公開するほうが安定します。

---

## 4.4 Part 4-4: 公開前チェックリスト

### 4.4.1 コンテンツ確認

- ☐ 章番号と見出しの並びが崩れていない
- ☐ `README.md` のリンク切れがない
- ☐ `Custom Agents` と `Skills` の用語が混同していない
- ☐ サンプル `.agent.md` が最小責務になっている

### 4.4.2 検証確認

- ☐ `npm install` が完了している

- `npm run ebook:build` が成功する
- 生成された EPUB の目視確認をした

### 4.4.3 公開準備

- `.gitignore` に生成物が含まれている
- `package.json` のスクリプトが README に記載されている
- 初回コミット用のメッセージを整理した
- 必要なら LICENSE / リポジトリ説明文を追加した

## 4.5 Part 4-5: FAQ

---

### 4.5.1 Q1. Custom Agent と Skill はどちらを先に作るべきですか？

初学者なら、まずは **小さな Custom Agent** を 1 つ作るのがおすすめです。

役割を分ける感覚がつかめたら、繰り返し手順を `Skill` に切り出すと整理しやすくなります。

---

### 4.5.2 Q2. `description` はどのくらい具体的に書くべきですか？

「いつ使うか」がわかる程度に具体的に書くのが理想です。

`README` , `documentation` , `bug` , `build` , `root-cause analysis` などの語を入れると伝わりやすくなります。

---

### 4.5.3 Q3. `tools` は多いほうが便利ですか？

必ずしもそうではありません。

多すぎると責務がばやけるため、まずは **必要最小限** が安全です。

なお `read` や `search` は `.agent.md` の中で自分で実装するものではなく、GitHub Copilot / VS Code 側のツールランタイムを使うための許可設定です。

---

#### 4.5.4 Q4. team で共有するときのコツは？

- 役割名を明確にする
- 命名規則をそろえる
- レビュー観点をチェックリスト化する
- 実際の利用例をもとに改善する

---

#### 4.5.5 Q5. この教材の検証コマンドは？

```
(#cb23-1)npm install
(#cb23-2)npm run ebook:build
```

これらはこのリポジトリで実行確認済みです。

# 第5章 Advanced Topics

---

## 5.1 Part 5-1: Subagents と Handoffs

---

### 5.1.1 なぜ必要か

教材作成や大きな保守作業では、1つの agent だけで全部やろうとすると責務が広がりすぎます。

そこで使う考え方が **subagent** と **handoff** です。

---

### 5.1.2 subagent とは

ある agent が、別の専門 agent に一部の仕事を任せる構成です。

例: - `tutorial-orchestrator` が全体を整理する - 文書整形は `docs-writer` - 問題調査は `debug-helper`

---

### 5.1.3 handoff とは

担当を切り替えるイメージです。

- 企画段階 -> docs agent
- 実装確認 -> debug agent
- 仕上げ -> repo-maintainer agent

---

## 5.1.4 設計のコツ

1. 親 agent の責務は **調整役** に絞る
  2. 子 agent には **明確な専門領域** を持たせる
  3. 循環的な受け渡しを避ける
  4. どこで最終判断するかを決める
- 

## 5.1.5 アンチパターン

- 全員が何でもやる
- handoff の条件が曖昧
- 親 agent 自身も実務を抱え込みすぎる

---

## 5.2 Part 5-2: チーム向け設計パターン

### 5.2.1 目的別の分け方

チームで運用する場合、agent は次のように分けると整理しやすくなります。

#### 5.2.1.1 ロール別

- docs-writer
- debug-helper
- repo-maintainer

#### 5.2.1.2 フェーズ別

- 企画用 agent
- 実装支援 agent

- レビュー用 agent

### 5.2.1.3 リスク別

- 読み取り中心の安全な agent
  - 編集可能な agent
  - 実行権限を持つ agent
- 

## 5.2.2 命名のコツ

- 短く、責務がわかる名前にする
- `helper`, `writer`, `reviewer`, `maintainer` など役割語を含める
- 曖昧な名前を避ける

悪い例: - `smart-agent` - `ultimate-helper`

良い例: - `docs-writer` - `debug-helper` - `frontend-reviewer`

---

## 5.2.3 レビュー観点

- 役割が一文で説明できるか
- `description` が具体的か
- ツール権限が過剰でないか
- 利用者が迷わず選べるか

## 5.3 Part 5-3: Orchestrator Agent の例

---

### 5.3.1 概要

`tutorial-orchestrator.agent.md` は、複数ステップの作業を整理し、必要に応じて専門 agent へ振り分ける **調整役** の例です。



### 5.3.1.1 ソースコード

```
---
description: "ドキュメントやリポジトリの作業が複数ステップにまたがり、
docs・デバッグ・保守などの役割調整が必要な場合に使うエージェント。複数ステッ
プの教材整備や調整作業に使います。"
tools: [read, search, todo, agent]
user-invocable: true
---
```

あなたは、複数ステップの作業を整理し、必要に応じて役割分担を判断する **\*\*調整役の Copilot agent\*\*** です。

#### ## 役割

- 大きな依頼を、進めやすい小さなステップに分解する
- 必要な場合だけ、より専門的な agent に作業を振り分ける
- 全体の進行状況を見失わないように整理する
- 利用者が次に何をすべきか分かる形で返す

#### ## 制約

- 理由なく別 agent に委譲しない
- 進展のない委譲ループを作らない
- 小さな作業を不必要に複雑化しない
- 専門 agent が不要な単純作業は自分で整理して進める

#### ## 進め方

1. まず依頼のゴールを一文で明確にする
2. 作業を 3～5 個程度の実行しやすいステップに分ける
3. 各ステップについて、自分で進めるか、専門 agent に任せるかを判断する
4. 必要なら委譲の理由を短く添える
5. 最後に、進捗・結果・次のアクションを簡潔にまとめる

#### ## 判断の目安

- ドキュメント改善が中心 -> `docs-writer`
- 原因調査や検証が中心 -> `debug-helper`
- リポジトリ整理や軽微な保守が中心 -> `repo-maintainer`

- 役割分担が不要な小作業 -> この agent がそのまま整理して進める

#### ## 出力形式

- **\*\*ゴール\*\***
- **\*\*ステップ一覧\*\***
- **\*\*委譲の有無と理由\*\***
- **\*\*現在の状況\*\***
- **\*\*次のアクション\*\***

### 5.3.1.2 `tutorial-orchestrator.agent.md` の設計・進め方（展開）

- **調整役のCopilot agent** として、複数ステップの作業を小さなタスクに分解し、必要に応じて専門agent（例：docs-writer, debug-helper, repo-maintainer）へ振り分けます。
- 依頼のゴールを一文で明確化し、3～5個の実行しやすいステップに分割。
- 各ステップごとに「自分で進めるか」「専門agentに任せるか」を判断し、委譲理由も簡潔に記載。
- 進捗・結果・次のアクションを整理して返します。

#### 5.3.1.2.1 判断の目安

- ドキュメント改善 -> docs-writer
- 原因調査・検証 -> debug-helper
- リポジトリ整理・保守 -> repo-maintainer
- 役割分担不要な小作業 -> orchestrator自身

#### 5.3.1.2.2 出力例

- ゴール
- ステップ一覧
- 委譲の有無と理由
- 現在の状況

- 次のアクション

---

このように、orchestrator agentは「全体の流れ設計と調整」に特化し、専門 agentとの役割分担を明確にする設計思想です。

この agent 自身が何でも実行するというより、「何を、どの順序で、誰に任せるとよいか」を見極めることに価値があります。

---

### 5.3.2 何をする agent か

- 大きな依頼を 3～5 個程度の進めやすいステップに分ける
  - 各ステップを、自分で進めるか専門 agent に任せるか判断する
  - 全体の進捗、判断理由、次のアクションを整理する
- 

### 5.3.3 判断の例

`tutorial-orchestrator` では、たとえば次のように役割分担を考えます。

- ドキュメント改善が中心 -> `docs-writer`
- 原因調査や検証が中心 -> `debug-helper`
- リポジトリ整理や軽微な保守が中心 -> `repo-maintainer`
- 単純な小作業 -> orchestrator 自身がそのまま整理して進める

このように、親 agent は調整に集中し、子 agent は専門性に集中するのが基本です。

---

### 5.3.4 向いている場面

- 教材リポジトリをまとめて整えたい
- docs と debug と maintenance が混ざる依頼
- 一度に複数の成果物や作業項目を整理したい

---

### 5.3.5 注意点

- 小さい依頼にまで orchestration を持ち込まない
- 子 agent の責務と親 agent の責務を混ぜない
- 理由のない委譲や、進展のないループ的 delegation を避ける

---

### 5.3.6 学び

高度な agent 設計では、**専門性の分割**と**調整役の明確化**が重要です。  
orchestrator は「全部やる agent」ではなく、**全体最適のために流れを設計する agent**と考えると理解しやすくなります。